Instituto Infnet

Quem recebe, valida

Projeto de Bloco: Inteligência Artificial Aplicada
Aula 12 · Etapa 6 · A API que recebe — o começo do TP4
Prof. Renan Santos

Onde paramos (Aula 11)

A v9 serve os dados a outros programas — sete rotas GET, tipadas, documentadas no /docs.

- **O bilhete: 'duas rotas GET para o SEU projeto'**
  Identifica o recurso? Caminho. Recorta a resposta? Consulta.
- **O tipo é a guarda**
  Sem ele, 200 e lista vazia; com ele, 422 e o motivo.
- **A prefeitura já busca os números direto**
  O código da Aula 2, apontado para a nossa API.

> **Até aqui o dado SAIU pela API. Hoje ele ENTRA — escrito por pessoas.**

🎬 Dá para a nossa API receber?

O pedido da gestora — e das equipes de campo da prefeitura.

- **"As equipes querem MANDAR coisas para o painel."**
  Fumaça na estrada, rio baixando, entulho no córrego — com UF e categoria.
- **"Elas já têm um celular com um formulário."**
  Ou seja: outro programa vai mandar dados para a nossa API — e o painel também.
- **O que muda quando a API RECEBE?**
  O dado vem de fora, escrito por pessoas: pode vir errado, incompleto, com o que não devia.

▸ **A pergunta da aula: como receber sem deixar entrar qualquer coisa — e sem dois programas brigando pelo mesmo arquivo?**

Onde estamos no Projeto de Bloco

Etapa 6, segunda metade — e o começo do TP4.



Objetivos da aula

Quatro capacidades — do corpo do POST ao painel que o envia.

- **Receber e validar**
  POST com um modelo Pydantic — e ler o 422 inteiro.
- **Guardar com segurança**
  Arquivo, trava, e o que pessoas enviam fora do Git.
- **Organizar e documentar**
  Routers, títulos, exemplos — o /docs como contrato.
- **Integrar o painel**
  O painel como cliente da API: pede, confere, e sobrevive à API desligada.

O que esta aula entrega ao TP4

O TP3 fechou com a Aula 11. Esta aula já é TP4 — e começa pela porta de entrada.

- **Item 2: "POST /processar_texto: rota que recebe um texto enviado pelo usuário"**
  Hoje: uma rota POST que recebe texto, validado por um modelo — o mesmo desenho, sem o LLM ainda.
- **Item 3: "os dados enviados e recebidos estejam no formato correto"**
  Hoje: os ENVIADOS (o modelo de entrada). Os RECEBIDOS — os response models — são a Etapa 7.
- **Item 4: "erros… feedback útil para o usuário final"**
  Hoje: o 422 traduzido no painel. As exceções HTTP (404, 503) são a Etapa 7; o modelo LLM, a 8.

▸ **[ASK] Que texto do SEU projeto o LLM do TP4 vai ler? Guardem a resposta para o fim.**

1 · Receber e validar

O primeiro POST: o corpo como dict

passso01_post_dict.py

```
relatos = []                # na memória, como o todo_list do livro

@app.post("/relatos")
def receber(relato: dict):     # o corpo do pedido vira este dicionário
    relatos.append(relato)
    return {"mensagem": "Relato recebido."}

# no GET, os dados iam na URL; no POST, vão no CORPO, em JSON
# a turma manda, pelo /docs: {"uf": 123} · {} · {"nome": "Maria"}
```

🐛 O POST que aceitava tudo

Tropeço real da v10 — e o livro admite: 'an empty dictionary could've also been sent' [Adeshina, p41].

- **{"uf": "PA", "texto": "Fumaça na estrada"} → 200**
  O que se esperava.
- **{"uf": 123} → 200 · {} → 200**
  UF número; relato vazio. Gravados.
- **{"nome": "Maria", "telefone": "91 9999-0000"} → 200**
  Dado pessoal, gravado — sem que ninguém pedisse.

▸ **Quem recebe, valida. A Aula 8 validava o CSV do usuário na borda; hoje a borda é a API.**

O modelo: o contrato do corpo

api/modelos.py

```
from pydantic import BaseModel, Field

class Relato(BaseModel):
    uf: SiglaUF                    # Literal com as 27 siglas
    categoria: Categoria           # Literal com 7 categorias
    texto: str = Field(min_length=15, max_length=500)

@router.post("")
def enviar_relato(relato: Relato):    # o tipo do parâmetro É o modelo
    ...
```

O 422 com a lista inteira

```
POST /relatos  {"uf": "Pará", "categoria": "fogo", "texto": "curto"}  ->  422

{"detail": [
  {"type": "literal_error",    "loc": ["body", "uf"],
   "msg": "Input should be 'AC', 'AL', …"},
  {"type": "literal_error",    "loc": ["body", "categoria"],
   "msg": "Input should be 'queimada', …"},
  {"type": "string_too_short", "loc": ["body", "texto"],
   "msg": "String should have at least 15 characters"}]}

# {} -> três "Field required"  ·  a função NEM roda
```

🐛 O nome e o telefone que 'foram gravados'

Tropeço real da v10: um relato válido + o nome e o telefone do morador que avisou.

- **Resposta: 200 — e 'nome' e 'telefone' SUMIRAM da resposta**
  O padrão do Pydantic é DESCARTAR em silêncio o que o modelo não declara.
  A equipe registrou: 'o sistema guarda o contato de quem avisou'. Não guardava.
- **O livro dá a entender o contrário: 'can only take four fields' [Adeshina, p40]**
  'Só aceita' quer dizer, por padrão, 'ignora o resto sem avisar' (verificado).
- **extra="forbid" → 422 · extra_forbidden, para nome E para telefone**
  'Extra inputs are not permitted'.

▸ **Recuse em voz alta o que você não guarda. Com dado pessoal, é também LGPD: quem envia precisa saber.**

A configuração do modelo

api/modelos.py · model_config

```
class Relato(BaseModel):
    model_config = ConfigDict(
        extra="forbid",              # campo a mais: 422, não descarte
        str_strip_whitespace=True,   # espaços das pontas não contam
        json_schema_extra={          # o exemplo que o /docs mostra
            "examples": [{"uf": "PA", "categoria": "queimada",
                          "texto": "Fumaça densa na BR-163…"}]},
    )

# 20 espaços → 422 string_too_short  ·  '  Fumaça…  ' → gravado sem eles
```

Livro × hoje — o Pydantic também mudou

O livro usa o Pydantic 1. O FastAPI de hoje, o Pydantic 2. Quatro divergências.

- **class Config: schema_extra = {...} [p51–54]**
  Roda, AVISA no terminal ('renamed to json_schema_extra') — e o exemplo SOME do /docs (passo 2).
- **'can only take four fields' [p40]**
  Por padrão, descarta o resto em silêncio. Recusar: extra='forbid'.
- **'field required' · 'value_error.missing' [p42] · .dict() [cap. 6]**
  Hoje: 'Field required' · 'missing' (+ o 'input' recebido) · .model_dump().

▸ **Quinta vez no semestre — e a primeira em que a fonte falha SEM ERRO: só um aviso no terminal. Leia o terminal.**

2 · Guardar e organizar

🐛 A memória que o --reload apagava

Tropeço real da v10. O livro: 'a temporary in-app database' [Adeshina, p37]. Temporária mesmo.

- **Passo 3: 3 POSTs em /memoria e em /arquivo → GET: 3 e 3**
  Salve QUALQUER .py — o --reload reinicia o servidor em ~3 s → GET: 0 e 3.
- **No projeto: data/relatos/relatos.csv, gravado por src/relatos.py**
  Modo 'a' (acrescentar) — 'w' zeraria tudo, o defeito 2 da A15. csv.DictWriter: vírgulas e aspas sobrevivem.
- **data/relatos/ no .gitignore**
  Conteúdo de pessoas não vai para repositório público. Sobem 5 exemplos FICTÍCIOS.

▸ **Memória de processo não é armazenamento. No Community Cloud, sumiria na primeira hibernação.**

🐛 Os relatos com o mesmo número

Cada relato ganha um número: o maior do arquivo + 1. Teste de carga: 40 envios ao mesmo tempo.

- **Sem trava: 9 · 6 · 9 números distintos em 3 rodadas (17 relatos com o nº 3)**
  E dos 120 envios, 117 gravados: alguns se atropelaram no arquivo.
- **Por quê? As rotas def rodam EM PARALELO**
  A escolha da Aula 11 (8,0 s × 2,0 s) cobra aqui.
- **Com uma trava (threading.Lock): 40 envios, 40 números**
  Um envio de cada vez — só durante a gravação, milissegundos.

▸ **A trava resolve para UM processo do servidor. Com vários, é trabalho para um banco de dados.**

Routers: o main.py vira roteador

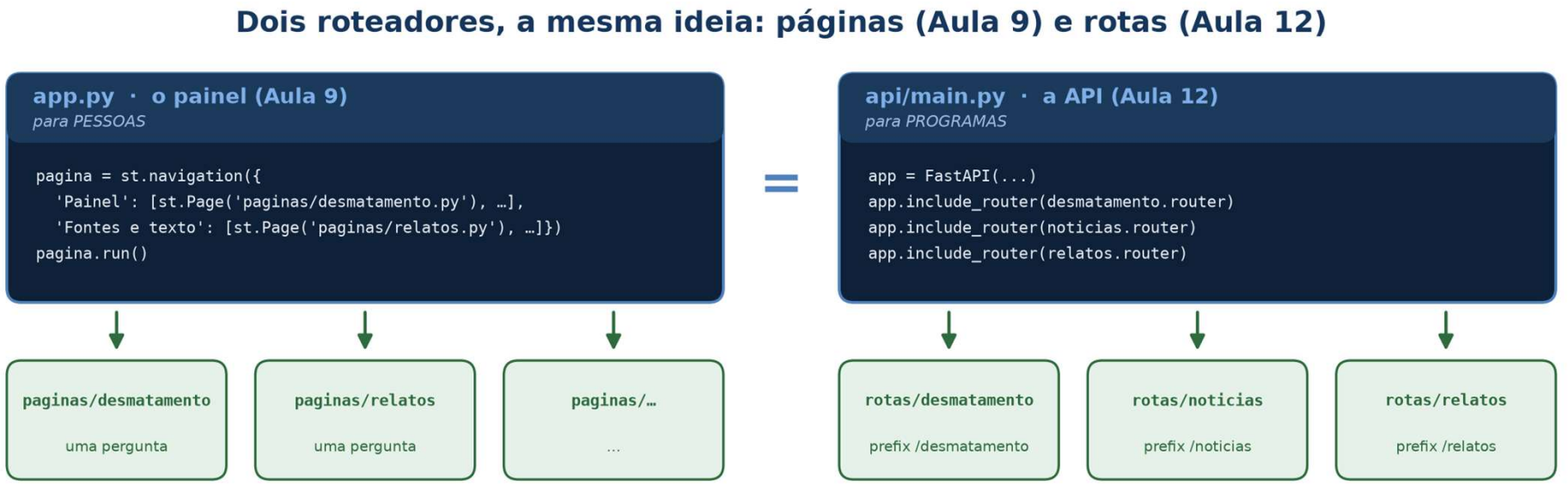
api/rotas/relatos.py + api/main.py

```
# api/rotas/relatos.py
router = APIRouter(prefix="/relatos", tags=["relatos"])
@router.post("", summary="Enviar um relato")
def enviar_relato(relato: Relato): ...

# api/main.py — cria o app e INCLUI os routers
app.include_router(desmatamento.router)
app.include_router(noticias.router)
app.include_router(relatos.router)
```

Dois roteadores, a mesma ideia

O app.py da Aula 9 e o api/main.py de hoje: o roteador não desenha nem responde — só diz quem existe.



O roteador não desenha nem responde: só diz quem existe. Um assunto, um arquivo — dos dois lados.

Documentar: o /docs é o contrato

Na v9, o /docs chamava as rotas de 'Serie Da Uf' e 'Apresentacao' — o nome da função.

- **summary='Série anual de uma UF' → o TÍTULO da rota**
  E a docstring da função → a DESCRIÇÃO: o que faz, o que recebe, o que devolve.
- **description= em Query/Path/Field → a explicação de cada parâmetro**
  tags= no router → o grupo; FastAPI(title=, description=, version=) → a capa.
- **json_schema_extra no modelo → o EXEMPLO no Try it out**
  E o 422 aparece documentado sozinho, em toda rota que valida.

▸ **A documentação é o contrato. Foi o que fez do IBGE o degrau 1 na Aula 2 — agora a de vocês.**

O /docs da v10

Captura real: o título (summary), a descrição (docstring) e o exemplo do modelo no corpo.



3 · Integrar: o painel como cliente

O painel pede — e confere

src/cliente_api.py · enviar_relato()

```
def enviar_relato(url, uf, categoria, texto):
    corpo = {"uf": uf, "categoria": categoria, "texto": texto}
    r = requests.post(f"{url}/relatos", json=corpo, timeout=10)
    if r.status_code == 422:              # a API recusou — e disse por quê
        return False, [traduzir(e) for e in r.json()["detail"]]
    r.raise_for_status()
    return True, r.json()["relato"]          # só então: "recebido"

# traduzir: ("texto", "string_too_short") -> "O texto precisa ter 15…"
```

🐛 O app que dizia 'enviado!'

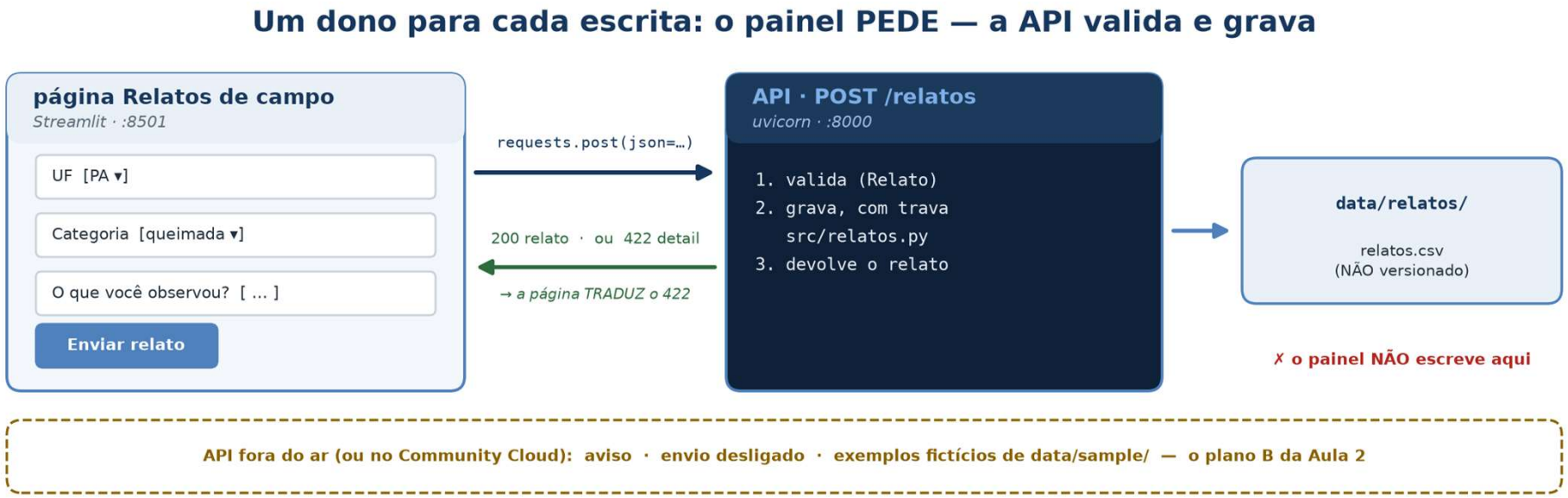
Tropeço real da v10: a primeira página de envio. Três decisões, três defeitos que elas evitam.

- **json= — e não data=**
  data= manda um FORMULÁRIO: 422 'Input should be a valid dictionary…'.
- **Conferir o status antes de dizer 'enviado'**
  Sem isso, a tela dizia 'Relato enviado!' a um envio que a API recusou — e nada era gravado. Nunca.
- **timeout= e a API desligada tratada (o plano B da Aula 2)**
  Sem isso: ConnectionError com o traceback inteiro na tela.

▸ **Quem recebe, valida; quem envia, CONFERE. O painel só diz 'enviado' quando a API disser que gravou.**

Um dono para cada escrita

O painel pede; a API valida e grava. O painel NÃO escreve no CSV.



O painel pede; a API decide. Se os dois escrevessem, a regra existiria em dois lugares — e um dia discordariam.

E no Community Cloud?

O app publicado procura a API em 127.0.0.1:8000 — que, lá, não tem API nenhuma.

- **A página avisa, desliga o envio e mostra 5 relatos de EXEMPLO**
  O plano B da Aula 2. Verificado localmente, com a API desligada; no Cloud, não testado.
- **O endereço é configuração, não código**
  api_url nos Secrets (ou PAINEL_ODS_API_URL) — a regra da Aula 5.
- **Publicar a API seria outro serviço**
  Fora do escopo do PB — e o TP4 também roda LOCAL, na sua máquina.

▸ **Um painel publicado não pode quebrar porque um serviço da SUA máquina está desligado.**

A19 · o código (os dois sobem sem erro)

api.py (uvicorn api:app --reload) · app.py (a página)

```
# api.py
relatos = []
@app.post("/relatos")
def receber(relato: dict):
    relatos.append(relato); return {"mensagem": "Relato recebido."}

# app.py
if st.form_submit_button("Enviar"):
    requests.post(API + "/relatos", data={"uf": uf, "texto": texto})
    st.success("Relato enviado!")
```

Ao vivo: a v10 nos dois terminais

terminal

```
uvicorn api.main:app --reload        # terminal 1 — a API
streamlit run app.py                 # terminal 2 — o painel

# 🪖 Relatos: um relato bom → 'Relato nº 1 recebido…'
#            'fumaça'       → 'O texto precisa ter pelo menos 15…'
# /docs: GET /relatos → o mesmo relato  ·  Ctrl+C na API → aviso + exemplos

git status        # data/relatos/ NÃO aparece (.gitignore)
git commit -am "v10: a API recebe relatos, routers e a página Relatos"
git push
```

O TP4, depois de hoje

O enunciado do TP4, item por item — o que a v10 já tem e o que vem.

- ✅ **Item 2, a rota: um POST que recebe TEXTO, validado por um modelo**
  POST /relatos tem o desenho do POST /processar_texto — falta o LLM atrás dela.
- ✅ **Item 3, metade: os dados ENVIADOS no formato certo · ✅ o painel integrado à API**
  A página Relatos pede, a API valida e grava; o 422 chega traduzido ao usuário.
- 🔜 **Etapa 7: os dados RECEBIDOS (response models) e os erros HTTP (404, 503)**
  Itens 3 e 4.
- 🔜 **Etapa 8: o modelo LLM local, escolhido com critérios — atrás da rota**
  Itens 1 e 2.

▸ **O TP3 fechou com a Aula 11. Daqui para a frente, tudo conta para o TP4.**

O texto que o LLM do TP4 vai ler

Os dados de hoje são o insumo do POST /processar_texto — e já chegam pela API.

- **Relatos de campo (GET /relatos)**
  A categoria bate com o texto? Urgência? Um resumo por UF.
- **Notícias (o texto completo da coleta)**
  Resumo automático; perguntas e respostas sobre a semana.
- **A série do desmatamento**
  Um parágrafo que explica os números: 'em 2025, o Pará…'.

▸ **Na Etapa 8, o LLM vira mais uma rota da mesma API — o POST /processar_texto do TP4.**

A ponte para a Etapa 7

Duas coisas que o /docs da v10 entrega, olhando de perto.

- **ranking/2030 → 200, 'linhas': 0**
  Não achar devia ser 404.
- **A resposta 200 do POST aparece como "string"**
  A API não declarou o FORMATO do que devolve. E criar algo devia responder 201.
- **É o capítulo 3 do livro [Adeshina, p66–68]**
  HTTPException, response_model, status_code=201 — a Etapa 7.

▸ **Hoje a API sabe o que ACEITA. Na Etapa 7, ela aprende a dizer, com precisão, o que RESPONDE.**

Anti-padrões de quem recebe dados

Quase todos respondem 200.

- **Corpo como dict · modelo sem extra='forbid' quando o dado é pessoal**
  Lixo gravado; nome e telefone descartados em silêncio — e alguém achando que estão salvos.
- **Guardar na memória · abrir em 'w' · gravar sem trava com rotas def**
  Some no reinício; apaga a coleta; números repetidos (9 em 40).
- **data= no lugar de json= · dizer 'enviado' sem conferir · sem timeout nem plano B**
  422 escondido; 'Relato enviado!' que não foi; traceback na tela.
- **O app escrever no mesmo arquivo que a API · versionar o que pessoas enviaram**
  Dois donos; dado pessoal num repositório público.

Princípios que levamos

Cinco frases que sobrevivem à próxima versão do Pydantic.

- **Quem recebe, valida**
  Com um modelo: confere, recusa com a lista inteira, documenta.
- **Recuse em voz alta o que você não guarda**
  Descartar em silêncio é o 200 que engana.
- **Um dono para cada escrita**
  O painel pede; a API grava.
- **Um assunto, um arquivo — e a documentação é o contrato**
  Dos dois lados: páginas e routers.

🎬 O caso, em 3 lições (v10)

O que a v10 ensinou, na ordem em que doeu.

- **Quem recebe, valida**
  O dict aceitava {} e {"uf": 123}; o modelo devolve os três erros de uma vez.
- **Recuse em voz alta o que você não guarda**
  O nome e o telefone 'gravados' que nunca foram.
- **Um dono para cada escrita**
  O painel que dizia 'enviado!' a um envio recusado — até conferir.

▸ **O TP4 começa aqui. Etapa 7: a API aprende a responder com precisão. Etapa 8: um LLM atrás de uma rota.**